PROPUESTA ACADÉMICA · FASE 3

Arquitectura de software

Fase 3: disponibilidad y evolución

Evolucionar una plataforma de citas médicas desde las decisiones de las fases 1 y 2.



FASE 3 · DECISIONES DE DISEÑO

Dos requisitos cambian el diseño

La evolución afecta al software y a la infraestructura.



Aplicación Flutter

Web y móvil comparten la API REST, la lógica de negocio, la autenticación, los permisos y las validaciones.



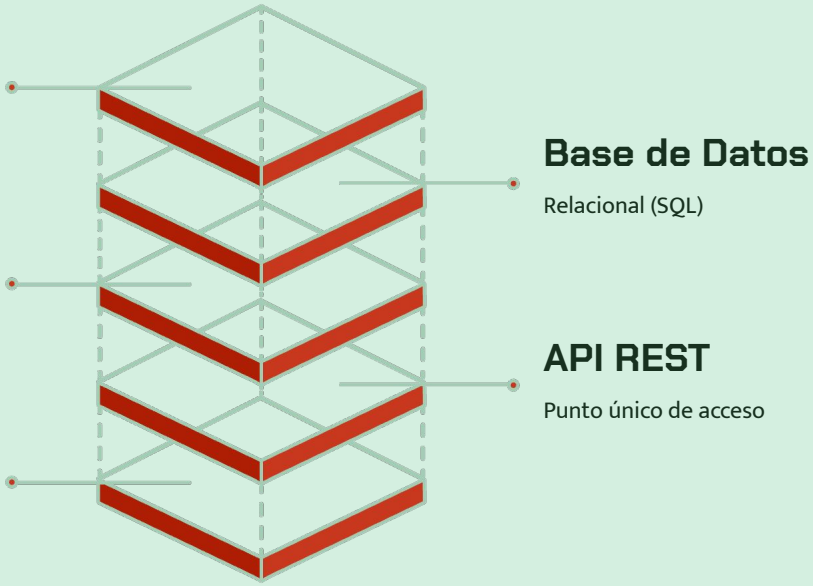
Disponibilidad 24/7

Se deben evitar puntos únicos de fallo y mantener las reservas durante mantenimientos y despliegues.

Consecuencia: también deben contemplarse la base de datos y las dependencias externas.

REUTILIZACIÓN DE LÓGICA

Web y móvil reutilizan la misma lógica



Una sola regla de reserva

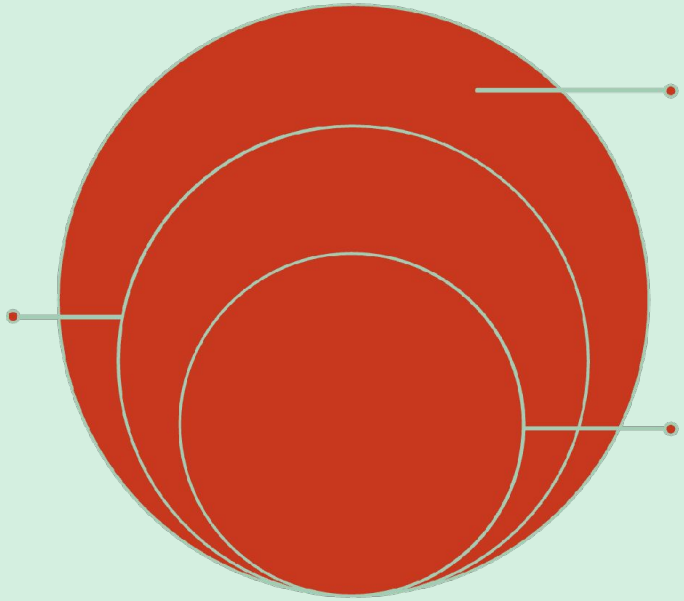
La API REST evita duplicar reglas de reserva, permisos y validaciones.

Backend como autoridad

La autenticación y la autorización se gestionan en el backend.

DISPONIBILIDAD PROPUESTA

Disponibilidad 24/7 sin detener las reservas



Observabilidad y Salud

Checks, monitorización y alertas tempranas

Despliegue Progresivo

Rolling deployments y control de versiones

Alta Disponibilidad

Balanceador, redundancia y DB replicada

Redundancia real

No basta con añadir servidores: la alta disponibilidad exige redundancia y configuración compatible.

Sesiones independientes

Las instancias no dependen de la sesión guardada en la memoria de una sola instancia.

PROPUESTA DE ARQUITECTURA

Arquitectura final: desacoplar donde aporta valor

Arquitectura propuesta

Entrada: Frontend web y aplicación móvil Flutter → balanceador → varias instancias del backend modular con API REST.

Módulos: usuarios, clínicas, médicos, citas, pagos, sincronización y notificaciones.

Datos e integraciones: las instancias conectan con una base de datos relacional de alta disponibilidad y con Stripe, Google Calendar y email.

Notificaciones: cola de mensajes → trabajador de fondo → servicio externo de email.

Decisiones de evolución

Monolito modular: Mantener usuarios, clínicas, médicos y citas dentro del monolito modular mientras no haya una razón clara para extraerlos.

Desacoplamiento selectivo: Encapsular Stripe y Google Calendar; desacoplar notificaciones con cola, trabajador, reintentos y manejo de errores si la infraestructura lo permite.

Consistencia: Proteger pagos y reservas contra duplicados y preservar la consistencia.

Lecciones: la arquitectura evoluciona con los requisitos; una API compartida evita duplicar lógica; la disponibilidad elimina puntos únicos de fallo; los módulos reducen el acoplamiento; los microservicios solo se justifican con beneficios concretos; la infraestructura importa tanto como el código.